Studio Nx

S1-10 프로젝트 전체 저장·불러오기 — 함수 설명 합본 (JS 269~273)

2026-10-19

목차

1. ai-notes/javascript/S1-10 - 프로젝트 전체 저장·불러오기/269_sequencer-commands.md
2. ai-notes/javascript/S1-10 - 프로젝트 전체 저장·불러오기/270_sequencer-history.md
3. ai-notes/javascript/S1-10 - 프로젝트 전체 저장·불러오기/271_sequencer-save.md
4. ai-notes/javascript/S1-10 - 프로젝트 전체 저장·불러오기/272_sequencer-init_load.md
5. ai-notes/javascript/S1-10 - 프로젝트 전체 저장·불러오기/273_sequencer-devices_full_notify.md

JavaScript 269 — 명령 레지스트리 · 커맨드 팔레트 (`sequencer-commands.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-commands.js`
작업	S1-10 프로젝트 전체 저장·불러오기: 메뉴·단축키·팔레트 공용 명령 레지스트리 (PRD 02 FR-2169)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	268 프로젝트 라우터(전체 조회 · 곡 설정) · 270 되돌리기 · 다시 하기

1. 하는 일

"무엇을 누르든 같은 명령 번호로 한 곳에서 처리한다"는 규칙을 만듭니다.

```
registerCommand(id, fn) / registerCommands({ id: fn })
                ↓
        registry  { id → { run, label, keys, group } }
                ↑
  메뉴(data-command) · 단축키 · 커맨드 팔레트(Ctrl+P) · Nyxi 액션
                ↓
           runCommand(id)
```

2. 언제 쓰나

쓰는 곳	
`sequencer-ui.js`	메뉴 항목 누르면 `runCommand(cmd)` 로 넘겨 줍니다
`sequencer-history.js`	`edit.undo` · `edit.redo` 를 여기에 등록합니다
`sequencer-save.js`	`project.save` 를 여기에 등록합니다
Ctrl+P	팔레트가 뜨고, 이름으로 검색한 뒤 Enter 로 실행합니다

3. 코드

```
const registry = {};
let cursor = 0; let rows = []; let palette = null; let toast = function () {};

export function registerCommand(id, run, meta) {
  if (!id || typeof run !== 'function') return false;
  const info = meta || {};
  registry[id] = {
    run: run,
    label: info.label || labelFromDom(id) || id,
    keys: info.keys || keysFromDom(id),
    group: info.group || 'command',
  };
  return true;
```

```
}

export function registerCommands(map, meta) {
  Object.keys(map || {}).forEach(function (id) { registerCommand(id, map[id], meta); });
}

export function runCommand(id) {
  const info = registry[id];
  if (!info) return false;
  info.run();
  return true;
}

export function initPalette(options) {
  if (options && typeof options.toast === 'function') toast = options.toast;
  registerCommand('view.palette', function () { openPalette(); }, {
    label: 'Command palette', keys: 'Ctrl+P', group: 'view',
  });
  document.addEventListener('keydown', function (e) {
    if (!(e.ctrlKey || e.metaKey) || e.shiftKey || e.altKey) return;
    if (e.key.toLowerCase() !== 'p') return;
    if (isTyping(e.target) || document.querySelector('dialog[open]')) return;
    e.preventDefault();
    openPalette();
  });
}

function filterPalette(query) {
  const q = String(query || '').trim().toLowerCase();
  rows = listCommands().filter(function (c) {
    if (!q) return true;
    return c.label.toLowerCase().indexOf(q) >= 0 || c.id.toLowerCase().indexOf(q) >= 0;
  });
  cursor = 0;
  if (!rows.length) {
    paletteList.innerHTML = '<li class="sq-palette__empty">No command matches.</li>';
    return;
  }
  paletteList.innerHTML = rows.map(function (c, i) {
    return '<li role="option" data-at="' + i + '" aria-selected="' + (i === cursor) + '">'
      + '<span>' + escapeHtml(c.label) + '</span>'
      + (c.keys ? '<kbd>' + escapeHtml(c.keys) + '</kbd>' : '')
      + '</li>';
  }).join('');
}
```

문법 풀이

식	뜻
`Object.keys(map || {}).forEach`	등록할 명령 여러 개가 들어 있는 사전을 돌면서 하나씩 등록합니다. `map` 이 없으면 빈 사전으로 봅니다
`registry[id] = { run, label, keys, group }`	한 명령이 무엇을 하는지, 어떤 단축키로 부르는지, 어떤 그룹인지 여기에 담습니다

식	뜻
`if (!info) return false`	아직 등록되지 않은 명령을 부르면 false 를 돌려줍니다 (부르는 쪽이 알림을 띄울 수 있습니다)
`e.preventDefault(); openPalette();`	Ctrl+P 를 잡고 열지 않아도 되게 한 뒤 팔레트를 띄웁니다
`isTyping(e.target)`	입력창에 커서가 있으면 단축키로 인식하지 않습니다
`escapeHtml(c.label)`	팔레트 글자를 넣기 전에 위험한 글자를 바꿔 둡니다
`li.dataset.at + rows[i]`	목록 줄 번호로 실제 명령을 찾아 실행합니다

4. 왜 이렇게

선택	이유
메뉴·단축키·팔레트가 같은 레지스트리를 씀	명령을 한 번만 등록하면 세 군데 모두에서 동작합니다 (FR-2169)
`runCommand` 가 false 를 돌려줌	아직 구현 안 된 명령이면 조용히 넘어가지 않고, 알림을 띄울 수 있습니다
팔레트가 등록된 것만 보여줌	아직 구현 안 된 명령을 목록에 띄우지 않습니다
`group` 칸	메뉴에서 카테고리별로 묶을 때 사용합니다. 팔레트에서는 label 순으로 정렬합니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 팔레트를 Ctrl+P 로 열고, 'Save' 로 검색한 뒤 Enter 로 저장하는 흐름, `runCommand('project.save')` 로 바로 부르는 흐름을 확인합니다. `initPalette` 가 한 번만 실행되고 `registerCommand('view.palette')` 가 겹치지 않는지도 봅니다.

JavaScript 270 — 되돌리기 · 다시 하기 (`sequencer-history.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-history.js`
작업	S1-10 프로젝트 전체 저장·불러오기: Undo/Redo (PRD 02 FR-255)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	269 명령 레지스트리 · 271 저장 · 자동 저장

1. 하는 일

디바이스(악기·이펙트)를 바꾼 동작을 기억하고, Ctrl+Z / Ctrl+Shift+Z 로 앞뒤로 넘깁니다. 최대 100단계까지 기억합니다.

```
pushHistory({ label, undo, redo })
        ↓
stack = [ 단계1, 단계2, …, 단계100 ]   ← at = 지금 위치
        ↓
undo()  → stack[at-1].undo()  → at -= 1
redo()  → stack[at].redo()    → at += 1
        ↓
applying = true 동안에는 새 이력을 쌓지 않습니다
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` `handleChange`	디바이스 추가·고치기·떼기·순서 바꾸기, BPM·Key 변경마다 `pushHistory` 를 부릅니다
메뉴 EDIT	`edit.undo` · `edit.redo` 명령으로 실행합니다
Ctrl+Z · Ctrl+Shift+Z · Ctrl+Y	`sequencer.js` `initShortcuts` 가 여기 `undo` · `redo` 를 부릅니다
커맨드 팔레트	`runCommand('edit.undo')` 로도 실행됩니다

3. 코드

```
export const HISTORY_LIMIT = 100;
let stack = []; let at = 0; let applying = false;
let onStateChange = function () {}; let toast = function () {};

export function initHistory(options) {
  const opts = options || {};
  if (typeof opts.onStateChange === 'function') onStateChange = opts.onStateChange;
  if (typeof opts.toast === 'function') toast = opts.toast;
  registerCommands({ 'edit.undo': undo, 'edit.redo': redo }, { group: 'edit' });
  notifyState();
}

export function pushHistory(entry) {
```

```
  if (applying || !entry || typeof entry.undo !== 'function' || typeof entry.redo !== 'function') return
false;
  stack = stack.slice(0, at);
  stack.push(entry);
  if (stack.length > HISTORY_LIMIT) stack = stack.slice(stack.length - HISTORY_LIMIT);
  at = stack.length;
  notifyState();
  return true;
}

export function undo() {
  if (!canUndo()) { toast('Nothing to undo'); return false; }
  const entry = stack[at - 1];
  applying = true;
  try { entry.undo(); } finally { applying = false; }
  at -= 1;
  notifyState();
  return true;
}

export function redo() {
  if (!canRedo()) { toast('Nothing to redo'); return false; }
  const entry = stack[at];
  applying = true;
  try { entry.redo(); } finally { applying = false; }
  at += 1;
  notifyState();
  return true;
}
```

문법 풀이

식	뜻
`stack.slice(0, at)`	되돌린 자리 뒤는 버립니다. 새로 바꾼 것을 기록하면 이전 되돌림 경로가 사라집니다
`applying = true` + `finally`	undo 를 실행하는 동안에 들어오는 알림(새 저장)을 무시합니다. 실패해도 반드시 풉니다
`stack.length > HISTORY_LIMIT`	100단계를 넘기면 가장 오래된 것부터 버립니다
`notifyState()`	메뉴의 `edit.undo` · `edit.redo` 항목을 지금 상태에 맞게 켜고 끕니다

4. 왜 이렇게

선택	이유
undo/redo 를 클로저로 담음	같은 동작을 다시 만들지 않고, 이전 값·이후 값을 함께 담습니다
100단계로 제한	FR-255 가 최소 100단계를 요구합니다. 메모리에 과하지 않게 상한을 둡니다
`applying` 깃발	undo 가 만든 알림이 다시 pushHistory 로 들어가면 이력이 어긋납니다
`canUndo()` · `canRedo()` 를 export	시험과 메뉴가 현재 상태를 알 수 있습니다
`historyLabels()` 는 시험용	어느 순서로 쌓였는지 눈으로 확인할 수 있습니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 이펙트 노브를 한 번 돌린 뒤 Ctrl+Z → 값이 이전으로 돌아가고, Ctrl+Shift+Z → 다시 이전 값이 되는지, `canUndo()` · `canRedo()` 가 그에 맞게 바뀌는지, 100단계를 넘기면 `historyDepth()` 가 100을 넘지 않는지 확인합니다.

JavaScript 271 — 저장 · 자동 저장 · 떠나기 경고 (`sequencer-save.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-save.js`
작업	S1-10 프로젝트 전체 저장·불러오기: 수동 저장 · 자동 저장 · 떠나기 경고 (PRD 02 FR-253, FR-256)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	270 되돌리기 · 다시 하기 · 272 시퀀서 시작과 전체 불러오기

1. 하는 일

BPM · Key 같은 곡 설정이 바뀌면 "저장할 것"으로 모아 두었다가 Ctrl+S 나 자동 저장 때 한 번에 보냅니다. 저장 안 된 것이 남아 있으면 창을 닫으려 할 때 브라우저가 한 번 물어봅니다.

```
queueSettings({ bpm: 140 })   → pending = { bpm: 140 }, dirty = true
        ↓
   30초 지나면(saveNow('auto')) 또는 Ctrl+S(saveNow('menu'))
        ↓
PATCH /projects/{id}/settings  ← pending 만 담아 보냅니다
        ↓
  성공 → pending 비우고 #save-state 칩을 ● Saved 로
  실패 → dirty 유지, 다시 시도
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` `handleChange`	`bpm` · `key` 변화를 `queueSettings` 로 넘깁니다
메뉴 FILE > Save	`project.save` 명령이 `saveNow('menu')` 를 부릅니다
Ctrl+S	`initShortcuts` 가 같은 `project.save` 명령을 실행합니다
자동 저장	마지막 변화 후 30초 뒤 `saveNow('auto')` 를 실행합니다
떠나기	`beforeunload` 이벤트가 저장 안 된 것이 있으면 경고를 띄웁니다

3. 코드

```
export const SETTINGS_KEYS = ['bpm', 'time_sig_num', 'time_sig_den', 'music_key', 'scale_mode', 'sample_
rate', 'master_settings'];
export const AUTOSAVE_MS = 30000;
let projectId = null; let dirty = false; let pending = {}; let saving = false;
let timer = 0; let autosaveMs = AUTOSAVE_MS; let toast = function () {};

export function initSave(options) {
  const opts = options || {};
  if (typeof opts.toast === 'function') toast = opts.toast;
```

```
  if (opts.autosaveMs) autosaveMs = opts.autosaveMs;
  if (opts.projectId !== undefined) setProjectId(opts.projectId);
  registerCommand('project.save', function () { saveNow('menu'); }, { label: 'Save project', group: 'fil
e' });
  registerCommand('project.discard', discardChanges, { label: 'Discard unsaved changes', group: 'file'
});
  window.addEventListener('beforeunload', onBeforeUnload);
  paintState();
}

export function queueSettings(patch) {
  let added = false;
  SETTINGS_KEYS.forEach(function (key) {
    if (patch && Object.prototype.hasOwnProperty.call(patch, key)) {
      pending[key] = patch[key];
      added = true;
    }
  });
  if (added) markDirty('settings');
}

export async function saveNow(reason) {
  if (!projectId) { if (reason === 'menu') toast('Saving needs the server'); return false; }
  if (saving) return false;
  if (!dirty) { if (reason === 'menu') toast('Everything is saved'); return true; }
  const body = Object.assign({}, pending);
  saving = true;
  try {
    await api.patch('/projects/' + encodeURIComponent(projectId) + '/settings', body);
    Object.keys(body).forEach(function (key) {
      if (pending[key] === body[key]) delete pending[key];
    });
    dirty = Object.keys(pending).length > 0;
    saving = false;
    paintState();
    if (reason === 'menu') toast('Saved');
    return true;
  } catch (err) {
    saving = false; dirty = true; paintState();
    toast(err && err.status === 403 ? 'You cannot save this project' : 'Could not save — we will try aga
in');
    if (!timer) timer = window.setTimeout(function () { timer = 0; saveNow('auto'); }, autosaveMs);
    return false;
  }
}

function onBeforeUnload(event) {
  if (!dirty) return undefined;
  event.preventDefault();
  event.returnValue = '';
  return '';
}
```

문법 풀이

식	뜻
`SETTINGS_KEYS`	서버가 받는 칸 이름 목록. 이 칸만 pending 에 넣습니다
`Object.prototype.hasOwnProperty.call(patch, key)`	`patch` 에 실제로 들어 있는 칸만 병합합니다
`Object.keys(body).forEach` + `pending[key] === body[key]`	저장하는 동안 다른 변화가 오면 그 칸은 남깁니다
`event.preventDefault()` + `returnValue = ''`	브라우저가 "정말 떠나시겠습니까?" 를 띄웁니다
`window.setTimeout(..., autosaveMs)`	마지막 변화 후 30초를 기다립니다. 중간에 다른 변화가 오면 `markDirty` 가 다시 예약합니다

4. 왜 이렇게

선택	이유
pending 모아서 한번에	노브를 빠르게 돌릴 때마다 PATCH 를 보내면 서버가 힘듭니다
30초 자동 저장	FR-253 이 자동 저장을 요구합니다. 매번 즉시 하지 않아도 되고, 오래 기다리지도 않습니다
`saving` 깃발	이미 저장 중일 때 다시 누르면 겹쳐서 보내지 않습니다
`beforeunload` 경고	FR-256. 저장 안 된 것이 있으면 브라우저 기본 확인 창을 띄웁니다
`paintState()`	상단 칩(#save-state)을 ● Saved / ● Unsaved changes / ● Saving… 으로 바꿉니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 BPM 칩을 바꾼 뒤 Ctrl+S 를 누르면 `• Saved` 가 되고 `PATCH /projects/{id}/settings` 가 갔는지, 자동 저장이 30초 안에 일어나는지(가짜 타이머로 확인), `beforeunload` 가 저장 안 된 상태에서만 경고를 띄우는지 확인합니다.

JavaScript 272 — 시퀀서 시작과 전체 불러오기 (`sequencer.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`
작업	S1-10 프로젝트 전체 저장·불러오기: init · loadProject · initShortcuts · handleChange (PRD 02 FR-253~255)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	271 저장 · 자동 저장 · 273 장치 전체 채우기와 이력 알림

1. 하는 일

시퀀서 화면의 시작점입니다. 명령·이력·저장을 먼저 세운 뒤, `GET /projects/{id}/full` 한 번으로 화면 전체를 채웁니다.

```
init()
 ├ initHistory · initSave · initPalette · initShortcuts
 ├ setDeviceNotifier(handleChange)  ← 장치 알림 → 이력·저장
 ├ readProjectId()  없으면 예시 화면 그대로
 ├ requireLogin()
 └ loadProject(projectId)
     ├ api.get('/projects/{id}/full')
     ├ applyProjectInfo(project)   ← BPM · Key · 박자
     ├ applyPatterns(patterns)     ← 왼쪽 패턴 목록
     └ applyFullProject(data)      ← 트랙·장치 그리기 (273)
```

2. 언제 쓰나

쓰는 곳	
`studioNx.html`	`<script type="module" src="static/js/pages/sequencer.js">` 가 자동으로 부릅니다
브라우저 주소	`studioNx.html?projectId=14431` 처럼 프로젝트 번호를 받습니다
메뉴·단축키	`runCommand(command)` 가 `edit.undo` · `project.save` 등을 레지스트리에 넘깁니다

3. 코드

```
export async function init() {
  initHistory({ toast: showToast });
  initSave({ toast: showToast });
  initPalette({ toast: showToast });
  initShortcuts();
  setDeviceNotifier(handleChange);

  const projectId = readProjectId();
  if (!projectId) return;
  setProjectId(projectId);
  if (!(await requireLogin())) return;
  try {
```

```
    await loadProject(projectId);
  } catch (err) {
    if (err instanceof ApiError && err.status === 401) return;
    showToast('Could not open this project — showing the example screen');
  }
}

export async function loadProject(projectId) {
  let data;
  try {
    data = await api.get('/projects/' + encodeURIComponent(projectId) + '/full');
  } catch (err) {
    if (err instanceof ApiError) {
      if (err.status === 404) { location.replace('errors/404.html'); return null; }
      if (err.status === 403) { location.replace('errors/403.html'); return null; }
      if (err.status >= 500) { location.replace('errors/500.html'); return null; }
    }
    throw err;
  }
  setProjectId(projectId);
  clearHistory();
  applyProjectInfo(data.project);
  applyPatterns(data.patterns);
  applyFullProject(data);
  return data;
}

export function handleChange(change) {
  if (!change || !change.type) return;
  switch (change.type) {
    case 'bpm': {
      const before = song.bpm;
      song.bpm = Number(change.bpm);
      pushHistory({ label: 'BPM ' + song.bpm, undo: () => applyBpmChange(before), redo: () => applyBpmCh
ange(song.bpm) });
      queueSettings({ bpm: change.bpm });
      break;
    }
    case 'deviceUpdate':
      pushHistory({
        label: change.label || 'Change device',
        undo: () => reapplyDevice(change.trackId, change.kind, change.deviceId, change.before),
        redo: () => reapplyDevice(change.trackId, change.kind, change.deviceId, change.after),
      });
      markDirty('device');
      break;
    default:
      break;
  }
}

export function initShortcuts() {
  document.addEventListener('keydown', function (event) {
    if (!(event.ctrlKey || event.metaKey) || event.altKey) return;
    if (event.target.closest && event.target.closest('input, textarea, select, [contenteditable]')) retu
rn;
    if (document.querySelector('dialog[open]')) return;
```

```
    const key = event.key.toLowerCase();
    if (key === 's' && !event.shiftKey) { event.preventDefault(); runRegistered('project.save'); }
    else if (key === 'z' && !event.shiftKey) { event.preventDefault(); undo(); }
    else if ((key === 'z' && event.shiftKey) || key === 'y') { event.preventDefault(); redo(); }
  });
}
```

문법 풀이

식	뜻
`readProjectId()`	`?projectId=14431` 에서 숫자만 꺼냅니다. 없으면 null (예시 화면)
`if (!projectId) return;`	예시 화면에서는 저장·이력을 만들지 않습니다 (S1-09 결정 유지)
`requireLogin()`	로그인 안 됐으면 `home.html#login?next=…` 으로 보냅니다
`ApiError` + `err.status`	404 · 403 · 500 은 각각 오류 페이지로 이동합니다
`clearHistory()`	프로젝트를 새로 열면 이전 이력을 버립니다
`song` 상수	BPM · Key 의 "지금 값". 되돌리기가 이전 값으로 돌아갑니다
`runRegistered('project.save')`	레지스트리의 `project.save` 를 실행합니다
`event.target.closest('input, …')`	입력창에 커서가 있으면 Ctrl+Z 를 무시합니다

4. 왜 이렇게

선택	이유
`init()` 이 항상 부르지 않음	`studioNx.html` 이 모듈을 불러오면 `init()` 이 자동으로 실행됩니다
`loadProject` 가 예외를 다시 던짐	404 · 403 · 500 은 오류 페이지로, 그 밖(네트워크 등)은 호출한 쪽이 처리합니다
`handleChange` 가 한 곳	화면 모듈(편곡·피아노롤·화면·장치)의 변화를 한 곳으로 모아 이력·저장을 붙입니다
`pushHistory` 안에서 `undo` / `redo` 정의	같은 값을 다시 넣는 함수를 바로 만들 수 있습니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 `?projectId=14431` 로 열면 `#bpm` 칩이 서버 BPM 값으로 바뀌고, `#pattern-list` 가 패턴 목록으로 바뀌고, `#tracks` 아래에 서버 트랙 줄이 들어가는지 확인합니다. Ctrl+S → 저장, Ctrl+Z → 이펙트 값 되돌리기, Ctrl+Shift+Z → 다시 하기를 확인합니다.

JavaScript 273 — 장치 전체 채우기와 이력 알림 (`sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-10 프로젝트 전체 저장·불러오기: applyFullProject · setDeviceNotifier · 이력 도우미 (PRD 02 FR-254~255)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	272 시퀀서 시작과 전체 불러오기 · (다음 작업) 클립 · 마커 저장

1. 하는 일

`/full` 로 받은 데이터로 트랙과 장치를 화면에 그린 뒤, 장치를 바꿀 때마다 `setDeviceNotifier` 로 연결한 곳(`sequencer.js` `handleChange`)에 알립니다.

```
applyFullProject(data)
 ├ tracks 배열 돌면서 renderTrack
 ├ instruments / effects 배열 돌면서 ensureDeviceRow
 ├ restoreOpenPanels — 열려 있던 칸 다시 열기
 └ open 칸이면 ensureLiveCells — 실시간 뷰 셀 채우기
        ↓
장치 추가 · 고치기 · 떼기 · 순서 바꾸기
        ↓
notify({ type: 'deviceAdd' | 'deviceUpdate' | 'deviceRemove' | 'deviceMove', … })
        ↓
sequencer.js handleChange → pushHistory + markDirty
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` `loadProject`	`applyFullProject(data)` 를 한 번 부릅니다
`sequencer.js` `init`	`setDeviceNotifier(handleChange)` 로 알림을 연결합니다
`moveEffect` · `addDevice` …	성공한 뒤 `notify(…)` 를 부릅니다
되돌리기 도우미	`reapplyDevice` · `dropDevice` · `restoreDevice` · `moveEffectSlots` 는 undo/redo 가 바로 부릅니다

3. 코드

```
let notify = function () {};

export function setDeviceNotifier(fn) {
  notify = typeof fn === 'function' ? fn : function () {};
```

```
}

export function applyFullProject(data) {
  const tracks = (data && data.tracks) || [];
  document.getElementById('tracks').innerHTML = '';
  tracks.forEach(function (track) {
    renderTrack(track);
    (track.instruments || []).forEach(function (inst) { ensureDeviceRow(track, 'instrument', inst); });
    (track.effects || []).forEach(function (fx) { ensureDeviceRow(track, 'effect', fx); });
  });
  restoreOpenPanels();
}

export function reapplyDevice(trackId, kind, deviceId, patch) {
  const track = trackById(trackId);
  const device = deviceById(track, kind, deviceId);
  if (!track || !device) return false;
  updateDevice(track, kind, device, patch, { silent: true });
  return true;
}

export function dropDevice(trackId, kind, deviceId) {
  const track = trackById(trackId);
  const device = deviceById(track, kind, deviceId);
  if (!track || !device) return false;
  removeDevice(track, kind, device, { silent: true });
  return true;
}

export function restoreDevice(trackId, kind, snapshot) {
  const track = trackById(trackId);
  if (!track) return false;
  const restored = addDevice(track, kind, snapshot, { silent: true });
  return Boolean(restored);
}

export function moveEffectSlots(trackId, from, to) {
  const track = trackById(trackId);
  if (!track) return false;
  moveEffect(track, from, to, { silent: true });
  return true;
}
```

문법 풀이

식	뜻
`document.getElementById('tracks').innerHTML = ''`	이전 화면을 비운 뒤 서버 순서대로 다시 그립니다
`notify({ type: 'deviceAdd', … })`	장치가 바뀐 사실을 시퀀서 시작점에 알립니다
`{ silent: true }`	되돌리기 도우미가 다시 `notify` 를 부르지 않도록 막습니다
`deviceById(track, kind, deviceId)`	트랙 안에서 장치를 찾습니다. 없으면 null
`Boolean(restored)`	addDevice 가 뭘 돌려주든 항상 true/false 로 맞춥니다
`trackById` export	되돌리기 도우미와 시험에서 트랙을 찾는 데 사용합니다

4. 왜 이렇게

선택	이유
`applyFullProject` 가 한 번에 다시 그림	부분 갱신은 코드가 복잡해지고, 서버가 이미 전부 줬으므로 그림을 맞추는 데 안전합니다
`notify` 는 모듈 안에 하나	`setDeviceNotifier` 로 시켜서 시작점이 연결합니다. 장치 모듈이 이력 모듈을 직접 알 필요가 없습니다
되돌리기 도우미를 export	undo/redo 가 같은 경로로 장치를 되돌립니다 (새로운 코드를 만들지 않음)
`silent` 플래그	되돌리기 중에 다시 이력이 쌓이면 어긋납니다. `history.js` `applying` 과 짝을 이룹니다
헤더 주석의 FR-2197 · 클립 불러오기는 유보	프런트에서 연결선 저장과 클립 불러오기는 이번에 넣지 않았습니다. 사용자 검토 때 다음 작업으로 제안합니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 `/full` 후 `#tracks` 아래 트랙 줄과 `data-device-id` 칸이 들어갔는지, 이펙트 순서를 바꾼 뒤 Ctrl+Z 로 이전 순서로 돌아오는지, 장치를 떼고 Ctrl+Z 하면 다시 돌아오는지, 되돌리기 중에 새 pushHistory 가 생기지 않는지 확인합니다.